OPTIMIZATION ALGORITHMS

Learning Objectives

- Apply optimization methods such as (Stochastic) Gradient Descent, Momentum, RMSProp and Adam
- Use random minibatches to accelerate convergence and improve optimization
- Describe the benefits of learning rate decay and apply it to your optimization

OUTLINE

* Mini-batch gradient descent
* Exponentially weighted averages
* Gradient descent with momentum
* RMSprop
* Adam optimization algorithm
* Learning rate decay
* The problem of local optima



deeplearning.ai

Optimization Algorithms

Mini-batch gradient descent

- We will see optimization algorithm that will help to train the neural networks much faster.

- Since machine learning is a highly empirical and iterative process, we have to train many models to try different parameters.

- However, training on a large dataset is slow.
- So, fast optimization algorithms may help to speed up the process.

Batch vs. mini-batch gradient descent

Vectorization allows you to efficiently compute on m examples.

Matrix of all training samples:

$$X = [x^{(1)}\ x^{(2)}\ x^{(3)}\ \dots\ x^{(1000)}\ x^{(1001)}\ \dots\ x^{(2000)}\ \dots\ \dots\ x^{(m)}]$$

(n_x, m)

$$Y = [y^{(1)}\ y^{(2)}\ y^{(3)}\ \dots\ y^{(1000)}\ y^{(1001)}\ \dots\ y^{(2000)}\ \dots\ \dots\ y^{(m)}]$$

$(1, m)$

- Vectorization allows to process all m examples quickly, but if m is large, it can still be slow.
- What if m is 5.000.000, 50 million or larger?

Andrew Ng

- In batch gradient descent we need to process the entire training set (5 million) before taking a little step of gradient descent.
- To train faster, we can split the training set into mini-batches. Let the mini-batch size be 1000.

$X = [\underbrace{x^{(1)}\ x^{(2)}\ x^{(3)}\ \dots\ x^{(1000)}}_{X^{\{1\}}:\text{mini-batch 1}}\ |\ \underbrace{x^{(1001)}\ \dots\ x^{(2000)}}_{X^{\{2\}}}\ |\ \dots\ |\ \underbrace{\dots\ x^{(m)}}_{X^{\{5000\}}}]$

(n_x, m) — $X^{\{1\}}$: $(n_x, 1000)$; $X^{\{2\}}$: $(n_x, 1000)$; $X^{\{5000\}}$: $(n_x, 1000)$

$Y = [\underbrace{y^{(1)}\ y^{(2)}\ y^{(3)}\ \dots\ y^{(1000)}}_{Y^{\{1\}}}\ |\ \underbrace{y^{(1001)}\ \dots\ y^{(2000)}}_{Y^{\{2\}}}\ |\ \dots\ |\ \underbrace{\dots\ y^{(m)}}_{Y^{\{1000\}}}]$

$(1, m)$ — $Y^{\{1\}}$: $(1, 1000)$; $Y^{\{2\}}$: $(1, 1000)$; $Y^{\{1000\}}$: $(1, 1000)$

- For 5M training data, and a mini-batch size of 1000 $\Rightarrow$ we will have 5000 mini-batches.
- Mini-batch t: $X^{\{t\}}, Y^{\{t\}}$

- Notation recap: $x^{(i)}$: i^{th} sample
  $z^{[l]}$: l^{th} layer

- In mini-batch gradient descent we process one $X^{\{t\}}, Y^{\{t\}}$ at a time rather than the entire training dataset.

- Mini-batch gradient descent runs much faster than batch gradient descent on a large dataset.

Mini-batch gradient descent

for $t = 1, \ldots, 5000$ {

Forward propagation on $X^{\{t\}}$

$$Z^{[1]} = W^{[1]} X^{\{t\}} + b^{[1]}$$

$$A^{[1]} = g^{[1]}(Z^{[1]})$$

$$\vdots$$

$$A^{[L]} = g^{[L]}(Z^{[L]})$$

} vectorized implementation (1000 examples)

1 step of gradient descent using $X^{\{t\}}, Y^{\{t\}}$ (as if $m = 1000$)

Compute cost $J^{\{t\}} = \frac{1}{1000} \sum_{i=1}^{1000} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_{l} \|W^{[l]}\|_F^2$ (from $X^{\{t\}}, Y^{\{t\}}$)

Backpropagation to compute gradients w.r.t $J^{\{t\}}$ (using $(X^{\{t\}}, Y^{\{t\}})$)

$$W^{[l]} := W^{[l]} - \alpha dW^{[l]}, \quad b^{[l]} := b^{[l]} - \alpha db^{[l]}$$

} ⇝ "1 epoch": 5000 batches (1 pass through training set)

Andrew Ng



deeplearning.ai

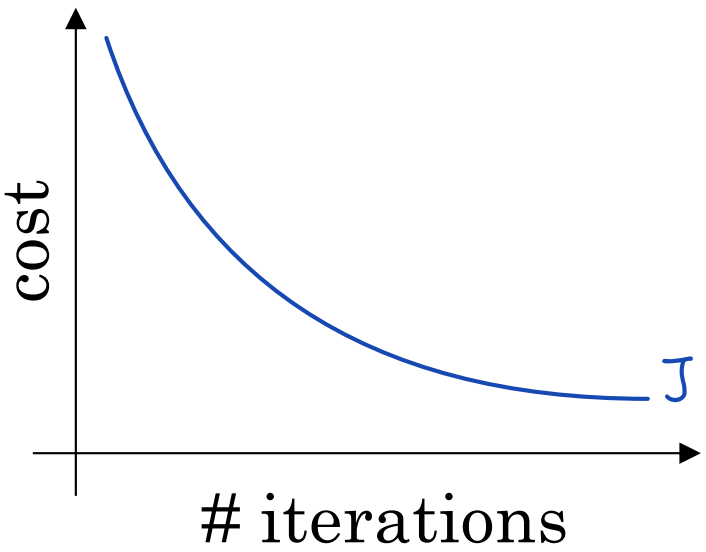
Optimization Algorithms

Understanding mini-batch gradient descent

- How to implement mini-batch gradient descent.
- What it is doing and why it works.

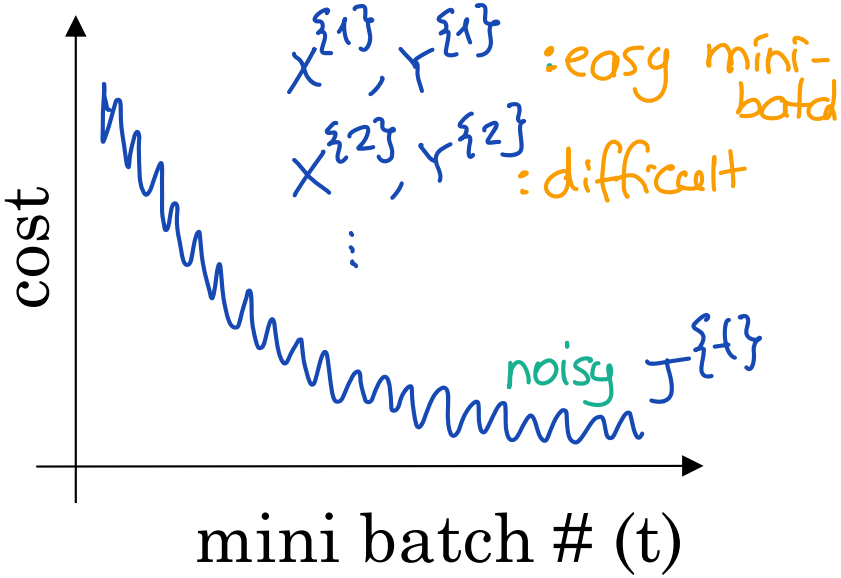
Training with mini batch gradient descent

Batch gradient descent



- Cost should decrease at each iteration.
- If it goes up, learning rate maybe too big.

Mini-batch gradient descent

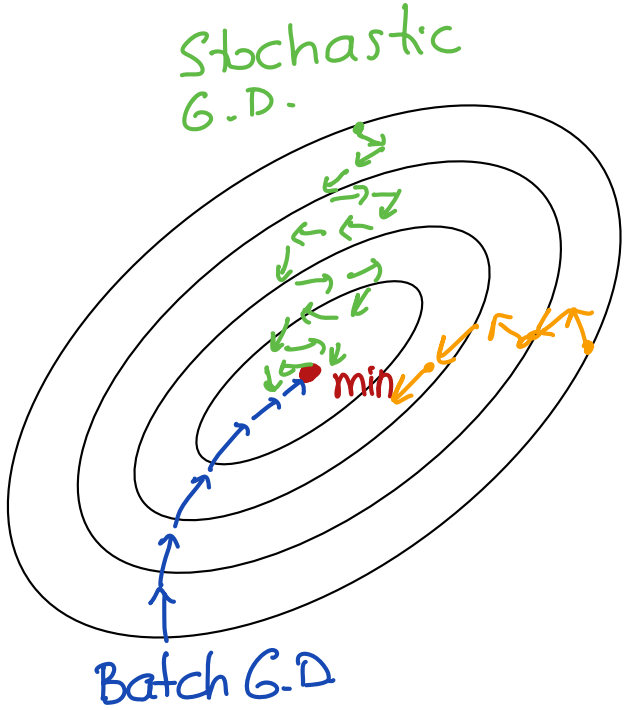


- Plot $J^{\{t\}}$ computed using $X^{\{t\}}, Y^{\{t\}}$
- Each mini-batch of data is different

Andrew Ng

Choosing your mini-batch size

- If mini-batch size=m : Batch gradient descent $(X^{\{1\}}, Y^{\{1\}}) = (X, Y)$
- If mini-batch size = 1 : Stochastic gradient descent. Every example is its own mini-batch: $(X^{\{1\}}, Y^{\{1\}}) = (x^{(1)}, y^{(1)})$, $(X^{\{2\}}, Y^{\{2\}}) = (x^{(2)}, y^{(2)})$
- In practice somewhere in-between 1 and m.



Andrew Ng

- Batch gradient descent converges to the minimum with relatively low noise.
  - mini-batch size=m → too long per iteration

- Stochastic gradient descent you may go in the wrong direction at some iterations. It is extremely noisy. On the average it will go in the correct direction. It won't ever converge, it will always oscillate and wander around the region of the minimum.
  - Noiseness can be reduced by using a smaller learning rate.
  - Huge disadvantage: You loose almost all your speed up from vectorization.

- In between: mini-batch size not too big or small.
  - Gives the fastest learning.
  - We get a lot of vectorization. Much faster than processing the examples one at a time.
  - Make progress without processing the entire training set.
  - May not converge, may oscillate around the minimum. (Remedy: to decrease the learning rate slowly, i.e. learning rate decay)

Choosing your mini-batch size

- If training set is small: use batch gradient descent ($m \leq 2000$)
- Bigger training set: Typical batch sizes are:

  64, 128, 256, 512 or 1024

  2^6 2^7 2^8 2^9 : powers of two (runs faster if a power of 2)

- Make sure mini-batch data fits in CPU/GPU memory

  $X^{\{t\}}, Y^{\{t\}}$



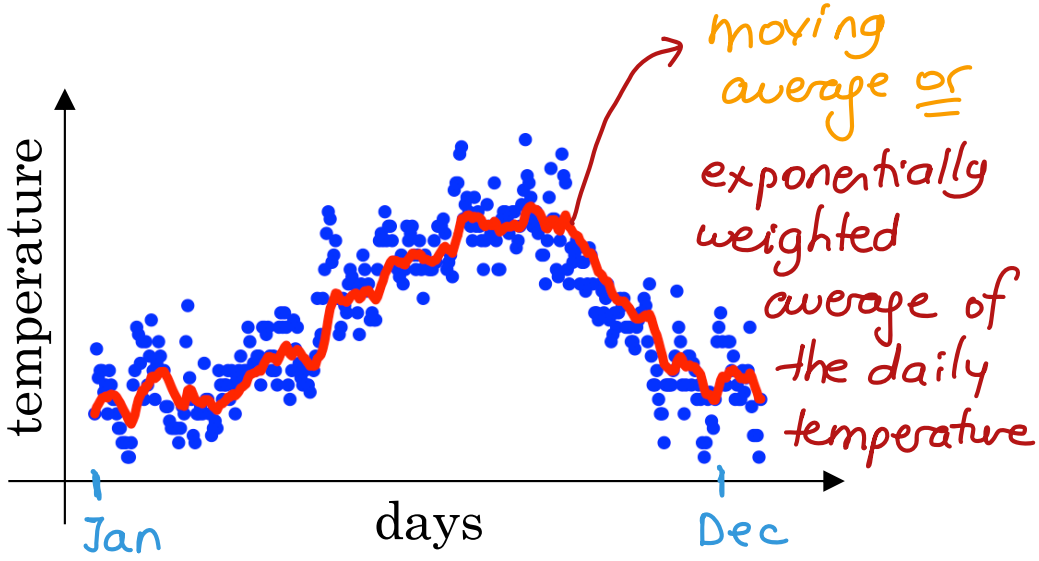
deeplearning.ai

Optimization Algorithms

Exponentially weighted averages

Temperature in London

$\theta_1 = 40°\text{F}$ 4°C
$\theta_2 = 49°\text{F}$ 9°C
$\theta_3 = 45°\text{F}$ ⋮
⋮
$\theta_{180} = 60°\text{F}$ 15°C
$\theta_{181} = 56°\text{F}$ ⋮
⋮



To compute the trends (local average):

$v_0 = 0$

$v_1 = 0.9 v_0 + 0.1 \theta_1$ → temperature on day 1

$v_2 = 0.9 v_1 + 0.1 \theta_2$ → temperature on day 2

$v_3 = 0.9 v_2 + 0.1 \theta_3$

⋮

Moving average of the temperature → $v_t = 0.9 v_{t-1} + 0.1 \theta_t$ → current temperature

→ average for previous day

Andrew Ng

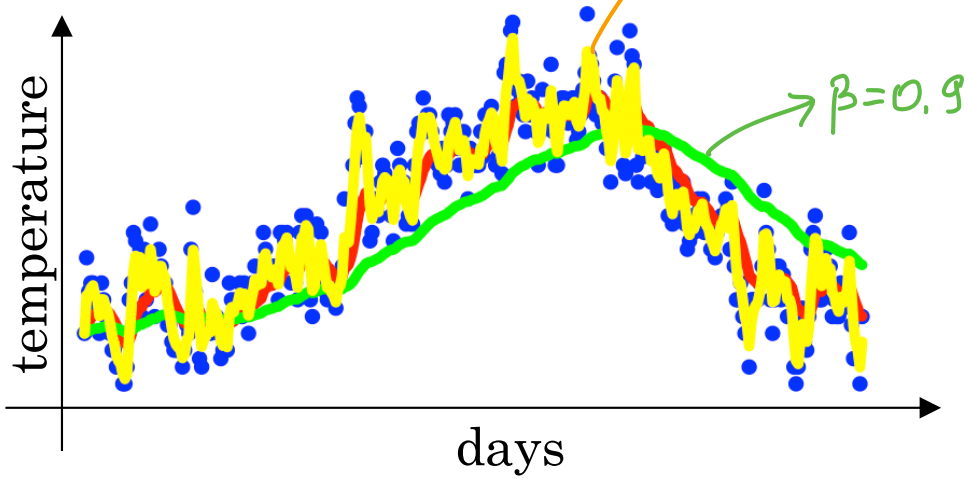
Exponentially weighted averages

$v_t = \beta v_{t-1} + (1-\beta)\theta_t$ → v_t is averaging over $\approx \frac{1}{1-\beta}$ days' temperature

$\beta = 0.9$: averaging over $\approx$ 10 days temperature

$\beta = 0.98$: $\frac{1}{1-0.98} = 50 \Rightarrow \approx 50$ " "

$\beta = 0.5$: $\frac{1}{1-0.5} = 2 \Rightarrow \approx 2$ days average



noisy but adapts to changes quickly.

$\beta = 0.98$, the curve is shifted to the right, i.e. has latency

- When β is close to 1, and when temperature changes daily, the exponentially weighted average adapts slowly.

Andrew Ng



deeplearning.ai

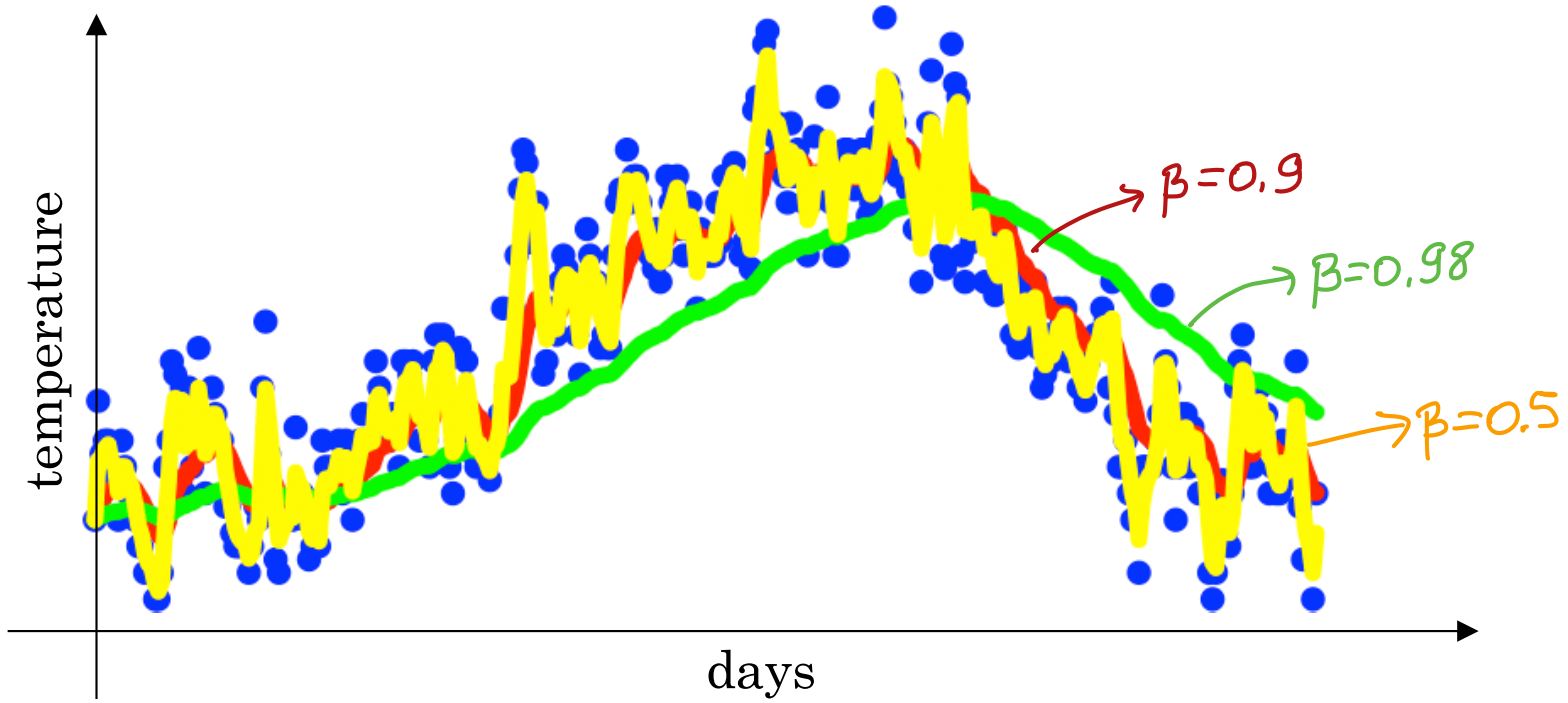
Optimization Algorithms

Understanding exponentially weighted averages

- Key component of several optimization algorithms in neural networks.

Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$



Andrew Ng

Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$
$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$
$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$
$$\dots$$

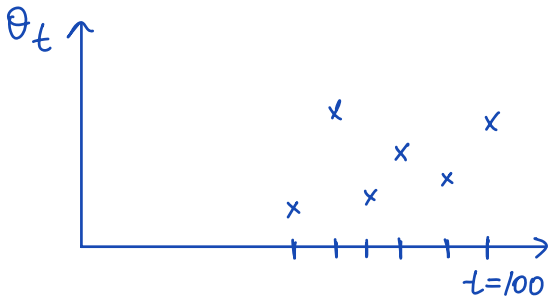
Analyze v_{100}:

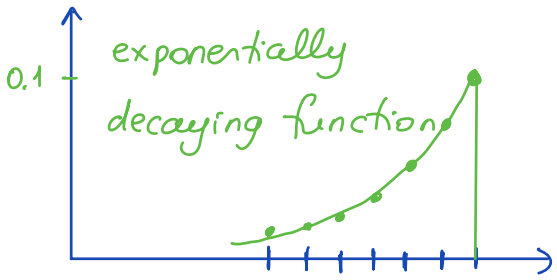
$$v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98}) \rightarrow (0.1\theta_{98} + 0.9v_{97})$$

(v_{99})

$$v_{100} = 0.1\theta_{100} + 0.1*0.9\theta_{99} + 0.1*(0.9)^2\theta_{98} + 0.1(0.9)^3\theta_{97} + 0.1(0.9)^4\theta_{96} + \dots (1)$$

Andrew Ng





- We take the daily temperature (θ_t) and multiply it with the decaying exponential element-wise and sum it up (see equation (1) above) to obtain v_{100}.

- How many days of averaging does this correspond to?

$\beta = 0.9$: $0.9^{10} \approx 0.35 \approx \frac{1}{e}$ $\Rightarrow$ it takes about 10 days for the peak of the exponential to decay one third of its value.

$(1-\varepsilon)^{1/\varepsilon} = \frac{1}{e}$

$\beta = 0.98$: $(0.98)^{50} \approx \frac{1}{e}$ $\Rightarrow$ averaging over ≈ 50 days of temperature.

$(1-\underbrace{0.02}_{\varepsilon = 1-\beta})^{50} \approx \frac{1}{e}$

- In general we are averaging over $\approx \frac{1}{1-\beta}$ days.

Implementing exponentially weighted averages

$v_0 = 0$

$v_1 = \beta v_0 + (1-\beta)\,\theta_1$

$v_2 = \beta v_1 + (1-\beta)\,\theta_2$

$v_3 = \beta v_2 + (1-\beta)\,\theta_3$

...

Implementation:

$v_\theta := 0$

$v_\theta := \beta v_\theta + (1-\beta)\theta_1$

$v_\theta := \beta v_\theta + (1-\beta)\theta_2$

⋮

- Very efficient in terms of computation and memory usage
- Used in machine learning

$v_\theta = 0$
Repeat {
  Get next θ_t
  $v_\theta := \beta v_\theta + (1-\beta)\theta_t$
}

takes very little memory (just v_θ and θ_t)

Andrew Ng



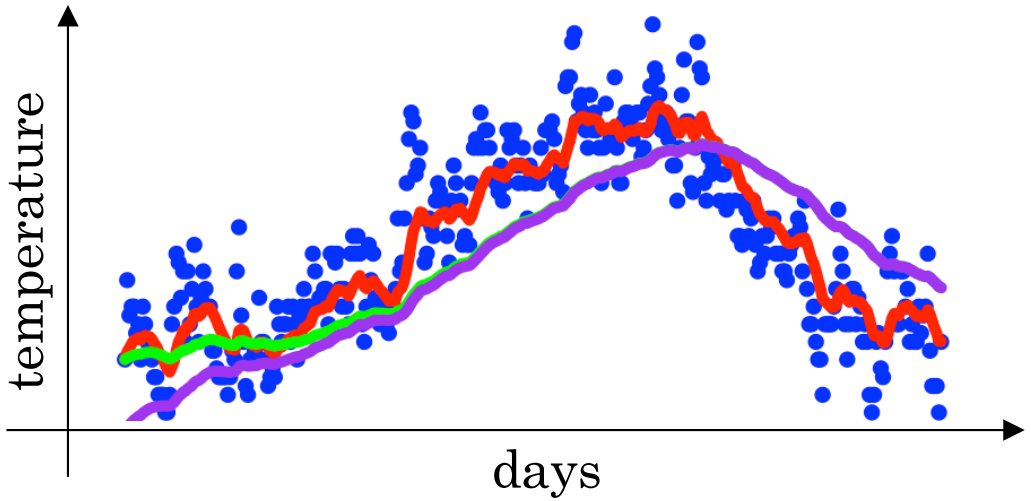
deeplearning.ai

Optimization Algorithms

Bias correction in exponentially weighted average

(to make the computation more accurate)

Bias correction



$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

- When $\beta = 0.98$ we don't get the green curve but we get the purple curve.
- But the purple curve starts really low. So, we need to fix that.

Start the computations from the beginning:

$v_0 = 0$

$v_1 = 0.98 v_0 + 0.02\theta_1$ (if $\theta_1 = 40 \Rightarrow v_1 = (0.02)\,40 = 0.8$: very low and not a good estimate of first days temperature)

$v_0 \to 0$

$v_2 = 0.98 v_1 + 0.02\theta_2$

$= (0.98)(0.02)\theta_1 + 0.02\theta_2$

$= 0.0196\theta_1 + 0.02\theta_2$ ⇝ v_2 will be much less than θ_1 or θ_2 and will not be a good estimate of the temperature of the first two days.

- We will modify it so that it becomes more accurate during the initial phase.

- Modification: Compute v_t as above and divide by $(1-\beta^t)$

$$\frac{v_t}{1-\beta^t}$$

$t = 2: \quad 1-\beta^t = 1-(0.98)^2 = 0.0396$

$$\frac{v_2}{0.0396} = \frac{0.0196\theta_1 + 0.02\theta_2}{0.0396}$$

(Notice: $0.0396 = 0.0196 + 0.02$)

weighted average of θ_1 and θ_2 and removes the bias.

- Note: When t is large $\beta^t \approx 0$ and $\frac{v_t}{1-\beta^t} \approx v_t$ and bias correction makes no difference.
- Bias correction is helpful during the initial phase (when t is small). We go from the purple line to the green line.
- We will use exponentially weighted moving averages to build better optimization algorithms.



deeplearning.ai

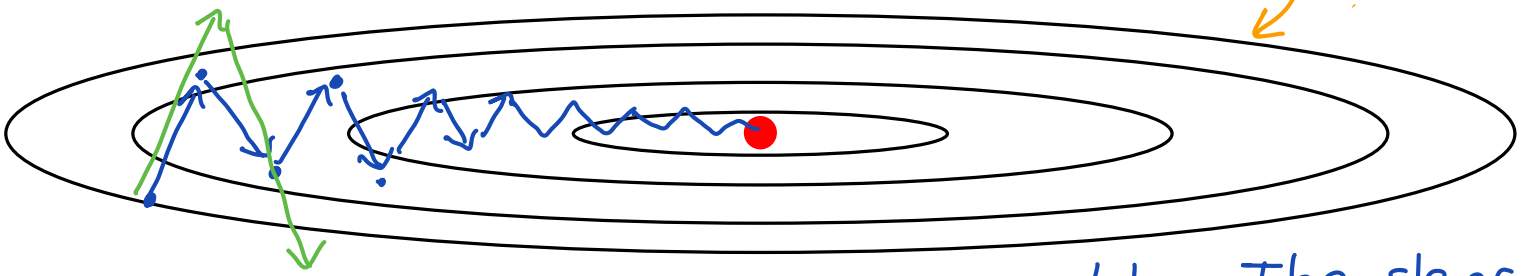
Optimization Algorithms

Gradient descent with momentum

- works faster than standard gradient descent.
- Computes the exponentially weighted average of the gradients to update the weights.

Gradient descent example

- Contours of the cost function.
- Red dot is the minimum



- The gradient descent steps are shown in blue. The steps may oscillate and slows down convergence. Also prevents using a large learning rate since you may overshoot and diverge (green lines).
- We want faster learning in horizontal direction and slower learning in vertical direction, since we want to move to the minimum.

Andrew Ng

- To improve we will implement <u>gradient descent with momentum</u> to smooth out gradient descent steps:

On iteration t:

Compute dW, db on current mini-batch

moving average of dW → $V_{dW} = \beta V_{dW} + (1-\beta) dW$ (Similar to $V_\theta = \beta V_\theta + (1-\beta)\theta_t$ before)

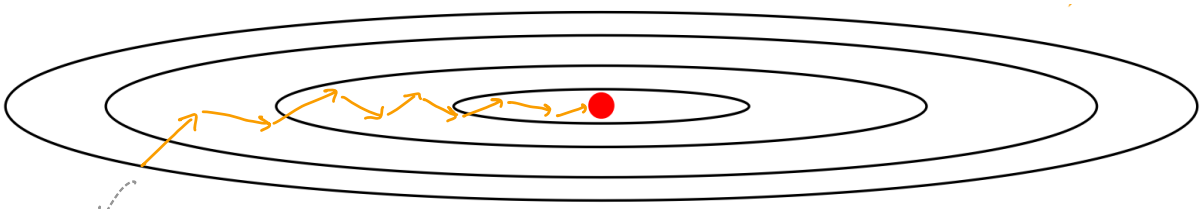
moving average of db → $V_{db} = \beta V_{db} + (1-\beta) db$

$W := W - \alpha V_{dW}$

$b := b - \alpha V_{db}$

- with momentum, the gradient steps will be smoother

↑↓ : positive and negative values will cancel out. in the vertical direction. The horizontal steps will be more directed to the minimum. Less oscillations will occur.



gradient descent with momentum has less oscillations and converges faster

Implementation details

$v_{dW} = 0$, $v_{db} = 0$

On iteration t:

Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

- In practice bias correction is not needed, since after 10 iterations moving average will warm up.

Hyperparameters: α, β $\beta = 0.9$: average over ≈10 days (most commonly used value)

↳ moving average parameter

↳ learning rate

Andrew Ng



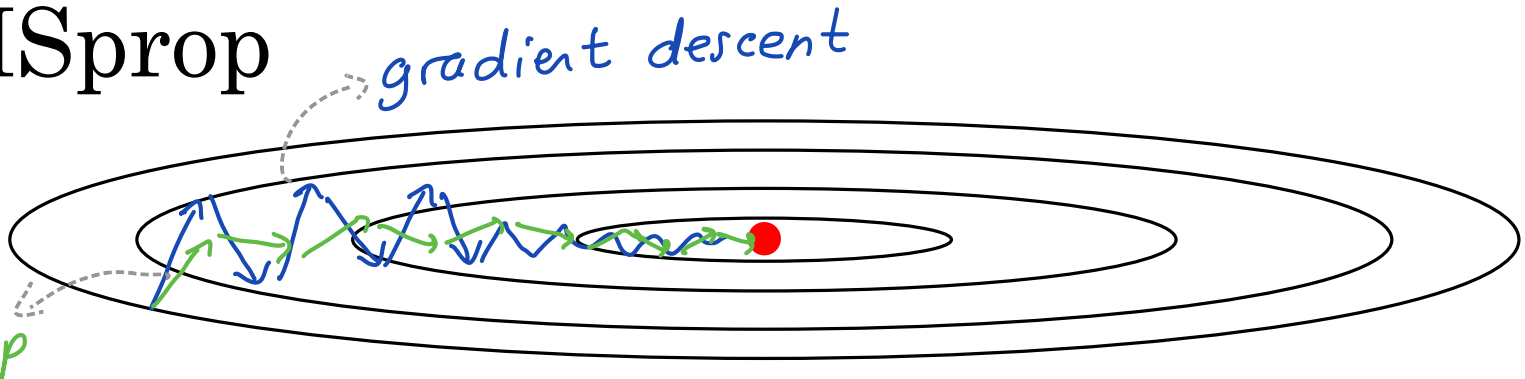
deeplearning.ai

Optimization Algorithms

RMSprop

- Root Mean Squared prop.
- Speeds up gradient descent

RMSprop



- Gradient descent has huge oscillations in the vertical direction. (blue path in the figure).
- Let's name the horizontal axis as w and the vertical axis as b:

for illustration:



- We want to slow down the learning or damp out the oscillations in the b direction, or in the vertical direction.
- We want to speed up learning or at least not slow it down in the horizontal direction.

Andrew Ng

- RMSprop will accomplish these:

On iteration t:

Compute dW, db on current mini-batch

$S_{dW} = \beta S_{dW} + (1-\beta) dW^2$ ⇢ elementwise square

⇢ moving average of squares of dW.

$S_{db} = \beta S_{db} + (1-\beta) db^2$

$W := W - \alpha \frac{dW}{\sqrt{S_{dW}}}$

$b := b - \alpha \frac{db}{\sqrt{S_{db}}}$

} parameter updates



- Intuition: The derivatives in the vertical direction (S_{db}) are "large" since there are large oscillations in b direction, and we are dividing db with a "large" number, in order to slow down the oscillations in b direction.
  - S_{dW} will be relatively small, so we are dividing dW with a relatively small number.
  - Net effect: updates will be smoother (green path in the figure above) in the vertical direction.
    ⇒ we can use a larger learning rate (α) and converge faster.
- It is called RMS: Root Mean Squared since we are squaring dW, computing the moving average and taking the square root to divide.

- Several modifications to the formulae above:

RMSprop will accomplish these:

On iteration t:

Compute dW, db on current mini-batch

$$S_{dW} = \beta_2 S_{dW} + (1-\beta_2) dW^2$$

$$S_{db} = \beta_2 S_{db} + (1-\beta_2) db^2$$

} Rename β as β_2

$$W := W - \alpha \frac{dW}{\sqrt{S_{dW} + \varepsilon}}$$

$$b := b - \alpha \frac{db}{\sqrt{S_{db} + \varepsilon}}$$

Add a small number to avoid division by zero.

$\varepsilon = 10^{-8}$

- Summary: RMSprop damper out large oscillations, and we can use a larger α (learning rate) for faster convergence.

- Next: Combine momentum and RMSprop to get a better optimization algorithm.



deeplearning.ai

Optimization Algorithms

Adam optimization algorithm

Adam optimization algorithm

- Combines momentum and RMSprop, which have been shown to work well in a wide range of deep learning architectures.
- Adam algorithm:

$V_{dw}=0$, $S_{dw}=0$, $V_{db}=0$, $S_{db}=0$ #initialization

On iteration t:

Compute dw, db using current mini-batch

#Compute momentum based moving averages

$V_{dw} = \beta_1 V_{dw} + (1-\beta_1) dw$

$V_{db} = \beta_1 V_{db} + (1-\beta_1) db$

} momentum with hyperparameter β_1

Andrew Ng

RMS prop-like update

$$S_{dw} = \beta_2 S_{dw} + (1-\beta_2) dw^2$$
$$S_{db} = \beta_2 S_{db} + (1-\beta_2) db^2$$

} RMS-prop-like update with parameter β_2

$$V_{dw}^{corrected} = \frac{V_{dw}}{(1-\beta_1^t)}, \quad V_{db}^{corrected} = \frac{V_{db}}{(1-\beta_1^t)}$$

$$S_{dw}^{corrected} = \frac{S_{dw}}{(1-\beta_2^t)}, \quad S_{db}^{corrected} = \frac{S_{db}}{(1-\beta_2^t)}$$

} bias correction

parameter update:

$$W := W - \alpha \frac{V_{dw}^{corrected}}{\sqrt{S_{dw}^{corrected} + \varepsilon}}, \quad b := b - \alpha \frac{V_{db}^{corrected}}{\sqrt{S_{db}^{corrected} + \varepsilon}}$$

- Adam algorithm has been shown to be very effective for a wide variety of architectures.

Hyperparameters choice:

α : needs to be tuned

work well in practice:

$\beta_1 = 0.9$ (for momentum-like term, weighted average of dw)

$\beta_2 = 0.999$ (for RMS-prop.like term, dw^2)

$\varepsilon = 10^{-8}$

Adam stands for Adaptive Moment Estimation.

Andrew Ng



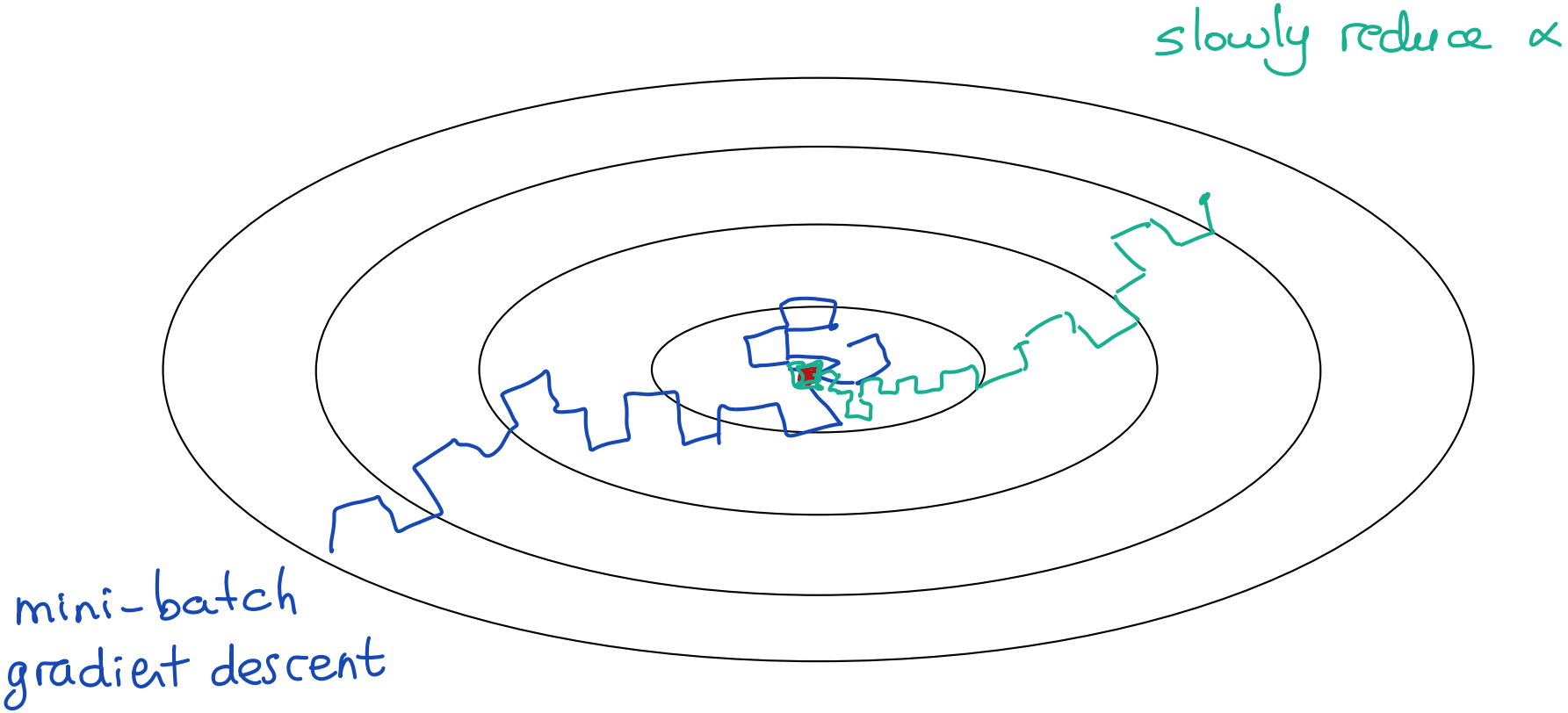
deeplearning.ai

Optimization Algorithms

Learning rate decay

- Slowly reducing the learning rate over time may speed up the learning algorithm.

Learning rate decay



Andrew Ng

- In mini-batch gradient descent the algorithm tends towards the minimum but it won't exactly converge. It may wander around the minimum point. (Blue path in the figure above)
- But if you slowly reduce your learning rate α, then during the initial steps, your α will be large and learning can be fast. As α gets smaller, the steps will be smaller and oscillations will be in a tighter region around the minimum.

Learning rate decay - Implementation

1 epoch = 1 pass through the data



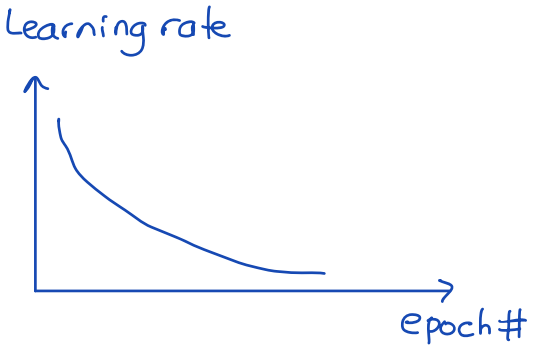
Learning rate decay:

$$\alpha = \frac{1}{1 + \text{decay_rate} \times \text{epoch_num}} \alpha_0$$

decay_rate: a hyperparameter to be tuned

if $\alpha_0 = 0.2$ and decay_rate = 1

Epoch	α
1	0.1
2	0.67
3	0.05
4	0.04
⋮	⋮



Other learning rate decay methods

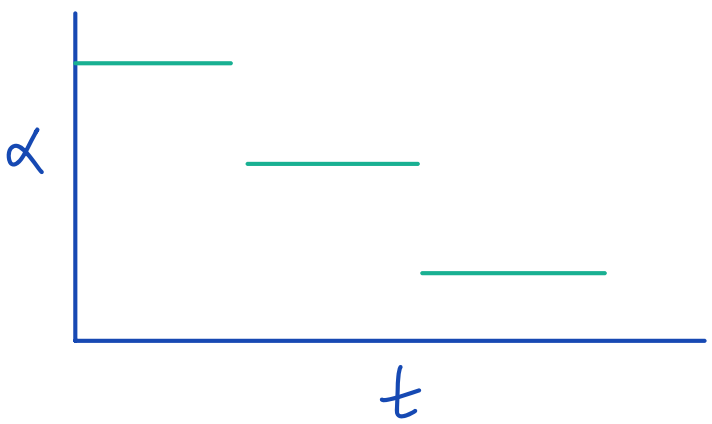
1) $\alpha = 0.95^{\text{epoch_num}} \cdot \alpha_0$ → exponential decay

2) $\alpha = \frac{k}{\sqrt{\text{epoch_num}}} \cdot \alpha_0$ or $\frac{k}{\sqrt{t}} \cdot \alpha_0$

t: mini-batch number

k: hyperparameter

3)



discrete staircase

Andrew Ng

4) Manual Decay: watch the model during training and decay manually.



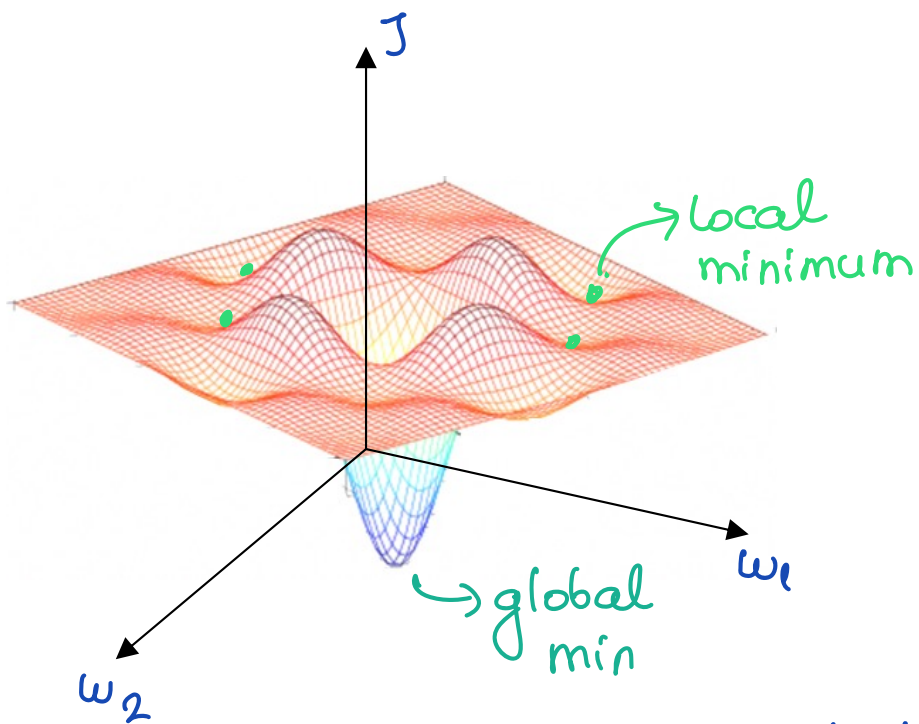
deeplearning.ai

Optimization Algorithms

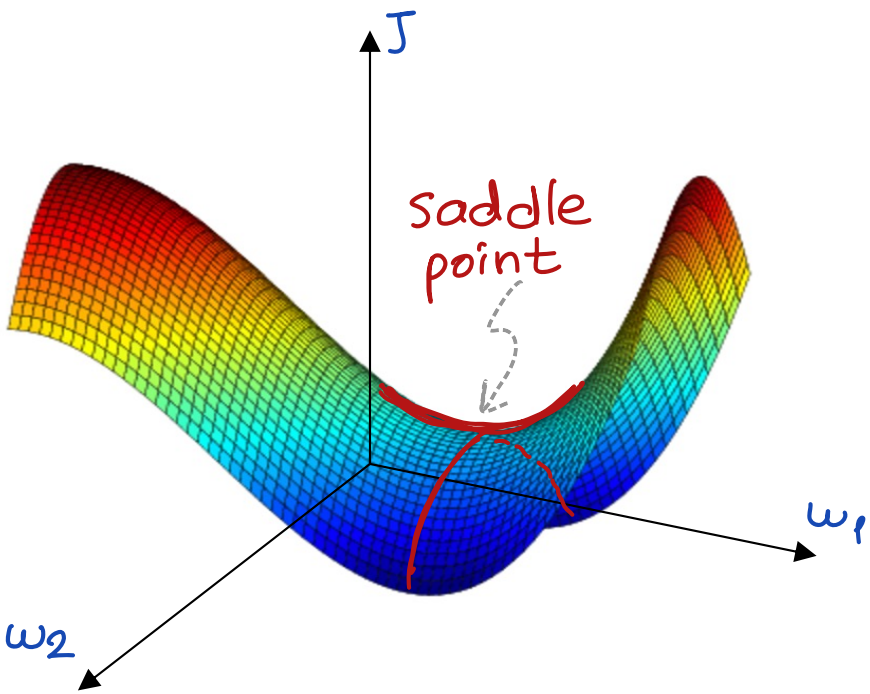
The problem of local optima

- In the early days of deep learning the optimization algorithm getting stuck in a local minimum was a concern.
- But as the theory of deep learning has advanced, our understanding of the local optima is also changing.

Local optima in neural networks



- Getting stuck in a local min. was a concern.



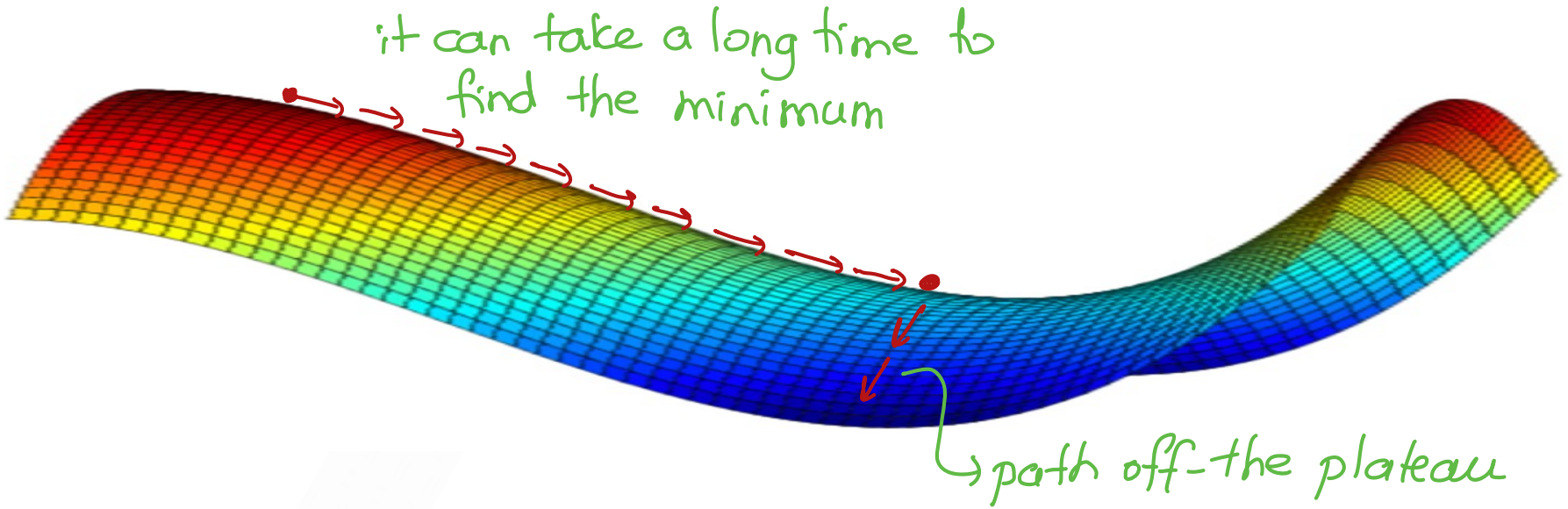
- It turns out that most points with zero gradients in a cost function are saddle points.
- Saddle point: Both a local min and a local max-point.

Andrew Ng

- Intuition: A function of very high dimensional space, if the gradient is zero, then in each direction it can either be a convex up function or a concave down function. ∪ ∩
  e.g. in 20000 space for the zero gradient pt. to be a local minimum 20000 of the dimensions should look like concave up. And the chance of that happening is very small. (2^{-20000})
  - In some directions the curve bends down, and in some other directions it bends up. ⇒ it is in general a saddle pt.

- Plateaus can really slow down learning.
- A plateau is a region where the derivative is close to zero for a long time.

Problem of plateaus



- Unlikely to get stuck in a bad local optima
- Plateaus can make learning slow: momentum, RMSprop and Adam can help the learning algorithm.

Andrew Ng